Dart and null safety

Null safety, Dart language basics, Flutter overview, Flutter toolchain

Lecturer
Dinu-Ștefan Rusu

Course
ADMD

What you should leave with

Null safety

Read `?`, `??`, `?.`, `!` and `late`, and know why the compiler can trust them.

Dart basics

Variables, functions, classes, collections, records and pattern matching.

Flutter overview

One Dart codebase, one renderer, and the widget catalog at a glance.

Flutter toolchain

Install, `flutter doctor`, hot reload versus hot restart, and a new project's layout.

1

Null safety

Nullable types, operators, late and required, Kotlin comparison

Contents Page

1 **Null safety** 3
2 Dart language basics 10
3 Flutter overview 22

Null reference errors

```
// Java. Compiles cleanly, no warning.
String name = findUser(42);
System.out.println(name.length());

// findUser is allowed to return null.
// NullPointerException at run time,
// on a user's device, in production.
```

Where null came from

Tony Hoare added the null reference to ALGOL W in 1965 because it was easy to implement. He later called it his ”billion-dollar mistake”.

Non-nullable by default arrived later, and separately, in Swift, Kotlin, Dart, and TypeScript’s strict mode.

The compiler can only help if the type says a value may be absent.

Limits of type safety

Compile-time type errors

The compiler rejects `int x = 'hi'` before it ever ships. Refactoring gets safer, at no runtime cost.

Runtime failures

Null, an out-of-range index, a request that fails. Null safety removes one whole family of these, not all of them.

Security vulnerabilities

A hard-coded key, a missing check, insecure storage. A fully type-checked app can still be completely insecure.

The one overlap is memory safety. Dart, Kotlin and Swift have no manual pointer arithmetic, so the classic exploitable bugs of C and C++, buffer overflows and use-after-free, cannot happen there. Security itself stays a design and review discipline; Lecture 10 comes back to it.

Dart null safety: the operators

```dart
// Non-nullable by default:
// String name = null; is an error.
String name = 'Ada';

// Add ? and null becomes allowed.
String? nickname;

// ?. calls only if non-null.
// ?? supplies a fallback.
final shown = nickname?.trim() ?? name;

// ??= assigns only when null.
nickname ??= 'anon';
```

Five symbols

- `?` the type may hold null
- `?.` reads or calls only if not null
- `??` supplies a fallback value
- `??=` assigns only when still null
- `!` asserts non-null, and throws if wrong

Sound null safety

```dart
// Flow analysis promotes a nullable
// to non-nullable within the branch.
String greet(String? name) {
  if (name == null) {
    return 'Hello, stranger';
  }
  return 'Hello, ${name.toUpperCase()}';
  // promoted: no ! needed here
}
```

Sound means the guarantee holds
A non-nullable Dart variable can never hold null at run time.

So the compiler stops emitting null checks for it. Sound null safety also makes release builds smaller and faster.

The late and required keywords

```dart
// late: promise to assign before use.
class Repo {
  late final Database db;
  Repo() { db = openDatabase(); }
}
// A broken promise throws
// LateInitializationError.

// required: a compile-time obligation.
String card({required String title}) {
  return title;
}
```

Two different promises

`late` defers assignment of a non-nullable field. Use it for dependency injection and expensive lazy fields, not to silence the analyzer.

`required` makes a named parameter mandatory. The caller cannot omit it, and the compiler checks that at every call site.

Null safety in Dart and Kotlin

Dart

```dart
String? findEmail(int id) {...}

void notify(int id) {
  final email = findEmail(id);
  if (email == null) return;
  send(email);
  // promoted to String
}
```

Kotlin

```kotlin
fun findEmail(id: Int): String? {}

fun notify(id: Int) {
  val email = findEmail(id)
  if (email == null) return
  send(email)
  // smart cast to String
}
```

Operators map almost one to one: `?.`, `??` (Kotlin: `?:`), `!` (Kotlin: `!!`), `late final` (Kotlin: `lateinit var`, `var` only).

2

Dart language basics

Variables, functions, classes, collections, records, pattern matching

Contents

		Page
1	Null safety	3
2	**Dart language basics**	10
3	Flutter overview	22

Variables: var, final, and const

```dart
// var infers its type from the value.
var count = 0;
String title = 'Lecture 2';

// final: assigned once, at run time.
final now = DateTime.now();

// const: computed by the compiler,
// baked into the binary.
const frameBudgetMs = 1000 / 120;

print('$title: $frameBudgetMs ms/frame');
```

Read it this way

- `var`: still statically typed, only inferred
- `final`: the binding cannot be reassigned
- `const`: a compile-time constant
- `dynamic`: opts out of checking, avoid it

Functions and named parameters

```dart
// Positional; => is a one-expr body.
int add(int a, int b) => a + b;

// Optional positional, with a default.
String tag(String n, [String s = '']) {
  return '$n$s';
}

// Named parameters: the Flutter style.
String card({
  required String title,
  String? subtitle,
}) => title;
```

Why named parameters matter
Every Flutter widget constructor uses them.

Functions are first-class values: pass them, store them, return them. That is what a callback like `onPressed` is.

Classes and constructors

```dart
class Lecture {
  final int number;
  final String title;
  final String? room;

  // this.x assigns the field directly.
  Lecture(this.number, this.title,
      [this.room]);
}

final l = Lecture(2, 'Languages');
```

The constructor is the field list

`this.x` in the parameter list assigns the field with no body needed.

Mark a field `final` and it must be set once, either at declaration or in the constructor, never after.

Getters, equality, and named constructors

```dart
class Lecture {
  final int number;
  final String title;

  Lecture(this.number, this.title);

  // Another named constructor.
  Lecture.lab(this.number)
      : title = 'Lab';

  // A getter: computed, no parens.
  String get label => 'L$number';
}
```

Equality is identity by default
Every class gets a default `toString`, `==`, and `hashCode`.

Two `Lecture` objects with the same fields are still not equal. Lecture 5 fixes that with Equatable.

Collections: List, Set, and Map

```dart
final topics = ['Dart', 'Kotlin'];
final seen = <String>{};
final points = {'exam': 3, 'lab': 3};

for (final t in topics) {
  seen.add(t);
}
```

Three built-ins

`List`, `Set` and `Map`. The angle brackets, where they appear, name the element types.

`where`, `map` and `forEach` are lazy `Iterable` operations; call `.toList()` when you need a `List` back.

Control flow and collection operators

```dart
// Collection-if / collection-for build
// a list inline, with no helper method.
final menu = [
  'Home',
  if (isAdmin) 'Admin',
  for (final t in topics) 'Lab: $t',
];

topics.where((t) => t.length > 4)
    .forEach(print);
```

Where you will see this
Flutter builds a list of children conditionally this way, without a helper function and without a temporary list.

`for`, `if` and `while` read like Kotlin and Java.

Records: typed, immutable tuples

```dart
// A record: an anonymous tuple type.
(String, int) parseLine(String line) {
  final parts = line.split(',');
  return (parts[0], int.parse(parts[1]));
}

final (name, score) =
    parseLine('Ada,97');
print('$name scored $score');
```

One type, many values
A record bundles values without declaring a class. Two records are equal if their fields are equal.

Named fields read like a tiny class: `({String title, int week})`.

Pattern matching: switch expressions

```dart
String describe(Object v) {
  return switch (v) {
    int n when n < 0 => 'negative',
    int() => 'a whole number',
    String s => 'text: $s',
    _ => 'something else',
  };
}
```

A switch that returns a value
Each arm is a pattern, an optional `when` guard, and an expression. `_` is the required wildcard.

The record from the previous slide can be destructured directly in a `case`, matching its shape and binding its fields at once.

Asynchronous code preview

One line for now

Dart handles a value that arrives later, with `Future`, `async` and `await`.

```
Future<String> fetchName(); // arrives later
```

Every example in this lecture is synchronous on purpose. The full model, the event loop, error handling and streams, is Lecture 4.

Dart syntax reference

Concept	Dart
Variable	`var count = 0;`
Immutable binding	`final now = DateTime.now();`
Compile-time constant	`const budget = 1000 / 120;`
Nullable type	`String? name;`
Required named parameter	`required this.title`
Typed list literal	`<String>['a', 'b']`

Kotlin and Dart compared

Dart

```dart
class User {
  final String name;
  final int age;
  const User(this.name, this.age);
}

final u = User('Ada', 36);
```

Kotlin

```kotlin
data class User(
  val name: String,
  val age: Int,
)

val u = User("Ada", 36)
```

Kotlin's `data class` gives you `==`, `hashCode`, `copy` and `toString` for free. In Dart you write them by hand, or generate them: Equatable in Lecture 5, freezed in Lecture 7.

3

Flutter overview

Rendering, widget catalog, JIT and AOT, hot reload, toolchain

Contents

		Page
1	Null safety	3
2	Dart language basics	10
3	**Flutter overview**	22

Flutter architecture

- Google’s open-source UI toolkit: one Dart codebase for Android, iOS, web, and desktop.
- Flutter does not reuse the platform’s own widgets. It ships its own widget library and draws every pixel itself, through the Impeller renderer.
- That is why a Flutter app looks identical on both platforms, and why native platform controls are not automatically available.

Your Dart code
widgets, state, logic

Flutter framework
layout, animation, gestures

Impeller
the renderer

GPU
Metal on iOS, Vulkan or OpenGL on Android

A release build is Dart, compiled ahead of time to native code. There is no interpreter and no JavaScript inside your app.

Widget catalog

Layout

Container, Padding, Row, Column, Stack: how you arrange pixels.

Input

TextField, Checkbox, ElevatedButton: what the user touches.

Scrolling

ListView, GridView: how a long list stays lazy.

Structure

Scaffold, AppBar, Drawer: the shell every screen sits in.

Feedback

SnackBar, AlertDialog, Tooltip: telling the user something happened.

Cupertino

An iOS-styled set of the same ideas. Lecture 3 goes deeper on all of this.

JIT and AOT compilation

Debug: JIT

`flutter run`

- The Dart VM compiles as it runs
- Hot reload swaps in new code in under a second
- Assertions on, DevTools attached

Release: AOT

`flutter build apk`

- Compiled ahead of time to native machine code
- No VM ships inside the app
- Fast cold start, tree-shaken, no hot reload

Never judge performance from a debug build. Measure in profile or release mode: `flutter run --profile`.

Hot reload versus hot restart

Hot reload
`r` in the terminal.

1. You save a `.dart` file
2. Changed source goes to the running VM
3. The VM re-JITs it in place
4. Flutter rebuilds the tree; state survives

What reload cannot do

- Change `main()` or start-up code
- Change an enum into a class
- Fix a compile error first
- Run inside a release build at all

Hot restart (`R`) rebuilds from scratch. All state is lost. Use it whenever `initState` or a global variable changed.

Installing the toolchain

```
# docs.flutter.dev/install

# Android Studio: SDK + emulator
# Xcode (macOS only): iOS builds
# VS Code or IntelliJ, with the
#   Dart and Flutter plugins

flutter doctor
# reports what is missing, per line
```

Run it first, always

`flutter doctor` names the missing piece. Read that line before asking for help.

An Android emulator is enough for every lab. A real device is faster, and the only way to test sensors.

iOS builds need macOS and Xcode; the labs never require them.

Creating and running a new app

```
flutter create my_app
cd my_app
flutter run
# debug: hot reload r, restart R
flutter run --release
# AOT machine code: what users get
```

This week's lab

Lab 2 walks the install end to end, on Windows, macOS, and Linux. `flutter create` scaffolds a runnable counter app. Read it before you change it.

The project structure of a new app

Path	What it holds
`lib/main.dart`	Entry point: `runApp()` and the root widget
`pubspec.yaml`	Dependencies, assets, and the SDK constraint
`android/`	The native Android project Gradle builds
`ios/`	The native Xcode project for iOS builds
`test/`	Unit and widget tests, run with `flutter test`
`build/`	Generated output, never committed to git

Everything you write lives under `lib/`. The rest is generated or native scaffolding you rarely touch.

Key numbers

1

Dart codebase, every platform

<1 s

typical hot reload turnaround

2

compilers: JIT for debug, AOT for release

Summary

1. **Null is part of the type.** `?`, `??`, `?.`, `!`, `late` and `required` remove a whole family of crashes.
2. **Dart's shapes map onto Kotlin.** Variables, functions, classes, collections, records and patterns: learn the concept once.
3. **Flutter draws every pixel itself.** JIT while you build for hot reload, AOT for what actually ships.

Further reading

dart.dev/language · dart.dev/null-safety · docs.flutter.dev/get-started · kotlinlang.org null safety

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel